CaseConnect Regulatory Platform

Complete Technical Workflow, Architecture Blueprint & Operational Reference Manual

CaseConnect: Technical Workflow & Architecture Documentation

Platform: CaseConnect — Centralized Institution Complaint & Regulatory Monitoring Platform

Target Solution: Statutory Regulatory Oversight & Inter-Bureau Coordination (Regulation Bureau, PGRC, Approval Bureau, Policy Bureau, Chief Vigilance Cell)

Database Architecture: Supabase PostgreSQL via the Node.js pg driver

Project Path: `C:\Users\Yuvraj\OneDrive\Desktop\Case Connect`

Generated Date: 2026-10-26

---

1. Executive Summary & Beginner Walkthrough

What is CaseConnect?

Technical colleges and universities are governed by regulatory bodies comprised of different cells:

- **Regulation Bureau (RB):** Ensures colleges maintain statutory faculty-student ratios, required laboratory space, and council norms.
- **Public Grievance Redressal Cell (PGRC):** Resolves complaints from students and parents regarding withheld certificates or capitation fees.
- **Approval Bureau (AB):** Oversees annual extension of approvals and sanctions.
- **Policy and Academic Planning Bureau (PAPB):** Handles autonomy and curricular benchmarks.
- **Chief Vigilance Cell (VIGILANCE):** Investigates financial malpractices and whistleblower petitions.

In traditional systems, each bureau worked in isolation. If a college faced a faculty salary complaint in PGRC and simultaneously applied for course expansions in AB, neither cell was aware of the other's docket.

CaseConnect unifies all complaints into a single regulatory console with automated cross-bureau overlap detection, SLA tracking, automated escalation, and 360° institutional compliance profiles.

The Beginner's End-to-End Walkthrough:

1. Starting Up: The official double-clicks `start.bat` or runs `npm start`. The server boots up at `http://localhost:5000`.

2. Accessing the Portal: The browser navigates to `http://localhost:5000`. The client-side router notices no authentication token is present in `localStorage` and immediately redirects to `#/login`.

3. Signing In: The user clicks any of the "Authorized Test Credentials" buttons (e.g., *Super Admin* or *PGRC Officer*) or enters credentials manually. The client sends a `POST /api/auth/login` request.

4. Receiving Credentials: The backend validates credentials using Bcrypt, generates a signed JWT token containing the user's role and bureau, and returns it. The browser stores this token in `localStorage`.

5. Dashboard Overview: The user lands on `#/dashboard`. A consolidated overview of active cases, ageing brackets (0–7 days, 8–15 days, 16–30 days, >30 days), bureau distribution, and alerts appears.

6. Registering a Case: An officer clicks **"Register New Case"**. They enter the institution's Permanent ID, title, description, and severity.

7. Automated Intelligence: When the case is saved, two automatic engines trigger:

- **SLA Calculation:** Computes deadline according to severity (e.g., Critical = 7 days, Low = 45 days).

- **Overlap Engine:** Scans other bureaus using Jaccard text similarity to identify matching allegations regarding the same college.

8. Inter-Bureau Coordination: If an overlap is flagged, reviewers can initiate a joint coordination docket or schedule joint hearings.

9. Resolution & Sign-off: As the inquiry progresses, the officer updates the case timeline, uploads remarks, and moves the case through the validated state machine (`RECEIVED` → `ASSIGNED` → `UNDER_PROCESS` → `DISPOSED` → `CLOSED`).

10. Signing Out: Clicking **"Logout"** purges the token from memory and `localStorage` , redirecting safely back to `#/login` .

---

2. System Architecture

CaseConnect is structured as a decoupled Single Page Application (SPA) served by an Express.js backend backed by Supabase PostgreSQL.

- **Client Tier (SPA):** HTML5, Vanilla ES6+ Modular Architecture, Standard CSS Design System & Clean Elevated Card System, Client Hash Router (`#/dashboard` , `#/cases` , `#/institutions`), Dynamic API Base URL Resolution (Zero 405 on Live Server / Static Hosting).
- **Server Tier (Node.js & Express):** Express.js Application Server (Port 5000), CORS Preflight & Global Security Headers (`nosniff` , `SAMEORIGIN` , `XSS block`), JWT Authentication Middleware & Role-Based Access Control (RBAC), 405 Method Not Allowed Interceptor & Safe JSON Error Responder.
- **Automated Engines:**

  - 3-Layer Overlap Detection Engine (Tokenization + Jaccard Similarity).

  - SLA Ageing & Escalation Engine (Periodic 30-min background interval).

  - Immutable Audit Trail Ledger Service.
- **Database Tier (PostgreSQL):** Supabase PostgreSQL through a private pg pool. Configure PGHOST, PGPORT, PGUSER, PGPASSWORD, PGDATABASE, and production CA verification.

---

3. Application Workflow (User Journey)

Stage 1: Initial Page Load

1. User loads `http://localhost:5000/` .

2. Browser downloads `frontend/index.html` , stylesheet `frontend/css/styles.css` , and core script modules.

3. `frontend/js/app.js` runs on `DOMContentLoaded` . It registers all hash routes and calls `Router.init('mainContentArea')` .

4. `Router.handleRoute()` evaluates `window.location.hash` . If no session token is stored, user is redirected to `#/login` .

Stage 2: Authentication

1. User inputs email and password on `#/login` .

2. `renderLoginPage()` intercepts form submission, calls `Auth.login(email, password)` .

3. `api.js` executes `POST /api/auth/login` .

4. On HTTP 200, JWT token and sanitized user profile are stored in `localStorage` keys `caseconnect_token` and `caseconnect_user` .

5. Router directs user to `#/dashboard` .

Stage 3: Dashboard & Case Management

1. User views real-time metrics loaded via `GET /api/reports/dashboard-stats`.
2. Active cases can be filtered by bureau, status, severity, or search query.
3. Officers can register cases (`POST /api/cases`), reassign officers (`PUT /api/cases/:id/assign`), or advance workflow state (`PUT /api/cases/:id/status`).

Stage 4: Cross-Bureau Overlap Adjudication

1. Reviewers navigate to `#/overlaps`.
2. The system surfaces flagged overlaps showing confidence scores, matched tokens, and case summaries.
3. The reviewer selects one of three determinations:

- `CONFIRMED_RELATED`: Cross-links case IDs in both records.
- `COORDINATION_INITIATED`: Creates an inter-bureau coordination docket.
- `DISMISSED_UNRELATED`: Marks the match dismissed with explanatory notes.

Stage 5: Sign-out

1. Clicking the user profile avatar dropdown triggers `Auth.logout()`.
2. Token and user profile are purged from `localStorage`.
3. Application navigates to `#/login`.

---

4. Role-Based Access Control & Workflows

Role Identifier	Role Display Name	Primary Scope	Permitted Actions
`SUPER_ADMIN`	Principal Monitoring Administrator	Global Council Governance	Full system read/write, user management, audit trail inspection, SLA parameter customization, benchmark seeding.
`SENIOR_MONITORING_OFFICER`	Senior Monitoring Director	Regulatory Supervision	Cross-bureau oversight, institution updates, audit log viewing, case re-assignment.
`BUREAU_OFFICER`	Bureau Scrutiny Officer	Bureau Operations (e.g., PGRC, RB)	Case registration, investigation timeline updates, status transitions, coordination initiation.
`REGIONAL_REVIEWER`	Regional Reviewer / Coordinator	Multi-Bureau Coordination	Overlap determination adjudication, cross-bureau ticket processing, institution dossier review.
`INSTITUTION_USER`	Institutional Representative	Institution Portal	Read-only view of 360° compliance dossier, submission of compliance remarks.
`AUDITOR`	Regulatory Compliance Auditor	Compliance Inspection	Read-only access to cases, reports, and immutable audit logs.

---

5. Login, Authentication & Session Security

Login Execution Steps

1. **Client Submission:** Form input is captured in `frontend/js/pages/login.js`. Form fields are validated (non-empty string, valid format).
2. **API Request:** Dispatched via `POST /api/auth/login` with payload `{"email": "...", "password": "..."}`.
3. **User Lookup:** Backend executes `User.findOne({ email: email.toLowerCase().trim() })`.

4. Password Verification: Bcrypt hashes the plaintext password against stored hash using `bcrypt.compare(password, user.passwordHash)`.

5. Token Generation: Upon match, `jsonwebtoken.sign()` produces a signed JWT with standard claims: `id`, `email`, `role`, `bureauId`.

6. Sanitized Response: Returns `{ success: true, token, user }` ensuring `passwordHash` is excluded.

7. Session Injection: Client sets `Authorization: Bearer <token>` on all future API requests via `ApiClient.request()`.

---

6. Complete API Endpoints Reference

Authentication Routes (`/api/auth`)

Method	Endpoint	Purpose	Request Body	Auth Required	Frontend Caller
`POST`	`/api/auth/login`	Official login	`{ email, password }`	Public	`login.js`
`POST`	`/api/auth/register`	Official account creation	`{ employeeId, fullName, email, password, bureauId, designation, phoneNumber }`	Public	`login.js`
`GET`	`/api/auth/me`	Verify active session	None	Bearer JWT	`auth.js`

Complaint Cases Routes (`/api/cases`)

Method	Endpoint	Purpose	Request Body	Auth Required	Frontend Caller
`GET`	`/api/cases`	List filtered cases	Query params (`status`, `bureauCode`, `severity`, etc.)	Bearer JWT	`cases.js`, `dashboard.js`
`GET`	`/api/cases/export`	Download cases CSV	Query params	Bearer JWT	`cases.js`
`GET`	`/api/cases/:id`	Case details	None	Bearer JWT	`caseDetails.js`
`POST`	`/api/cases`	Register new case	`{ permanentId, institutionName, bureauCode, category, title, description, severity, priority, petitioner }`	Bearer JWT	`modals.js`
`PUT`	`/api/cases/:id/status`	Advance status	`{ status, remarks }`	Bearer JWT	`caseDetails.js`
`PUT`	`/api/cases/:id/assign`	Assign officer	`{ officerId, officerName }`	Bearer JWT	`caseDetails.js`
`POST`	`/api/cases/:id/remarks`	Add timeline note	`{ note }`	Bearer JWT	`caseDetails.js`
`POST`	`/api/cases/:id/escalate`	Manual escalation	`{ reason }`	Bearer JWT	`caseDetails.js`

Institutions Routes (`/api/institutions`)

Method	Endpoint	Purpose	Request Body	Auth Required	Frontend Caller
`GET`	`/api/institutions`	Search institutions	Query params (`query` , `region` , `institutionType`)	Bearer JWT	`institutions.js`
`GET`	`/api/institutions/:id`	Single institution	None	Bearer JWT	`institutions.js`
`GET`	`/api/institutions/:id/profile-360`	360° Consolidated Dossier	None	Bearer JWT	`institutionProfile.js`
`POST`	`/api/institutions`	Register institution	`{ permanentId, name, region, state, city, ... }`	Super Admin / SMO	`modals.js`
`PUT`	`/api/institutions/:id`	Update institution	Partial metadata object	Super Admin / SMO	`institutionProfile.js`

Bureaus & Coordination Routes (`/api/bureaus`)

Method	Endpoint	Purpose	Request Body	Auth Required	Frontend Caller
`GET`	`/api/bureaus`	Bureau workload & capacity	None	Bearer JWT	`bureaus.js` , `workload.js`
`GET`	`/api/bureaus/coordination`	Coordination dockets	None	Bearer JWT	`bureaus.js`
`POST`	`/api/bureaus/coordination`	Initiate coordination ticket	`{ requestingBureauCode, targetBureauCode, primaryCaseId, subject, justification }`	Bearer JWT	`modals.js`
`POST`	`/api/bureaus/coordination/:id/action`	Update coordination docket	`{ status, note }`	Bearer JWT	`bureaus.js`

Overlap Engine Routes (`/api/overlaps`)

Method	Endpoint	Purpose	Request Body	Auth Required	Frontend Caller
`GET`	`/api/overlaps`	List overlap reviews	Query params (`status` , `bureauCode`)	Bearer JWT	`overlaps.js`
`GET`	`/api/overlaps/check/:caseId`	On-demand overlap scan	None	Bearer JWT	`caseDetails.js`
`POST`	`/api/overlaps/:id/review`	Record review decision	`{ determination, reviewerNotes }`	Authorized Reviewer	`overlaps.js`

Notifications & System Routes

Method	Endpoint	Purpose	Request Body	Auth Required	Frontend Caller
`GET`	`/api/notifications`	Unread alerts	None	Bearer JWT	`header.js`, `notifications.js`
`PUT`	`/api/notifications/:id/read`	Mark read	None	Bearer JWT	`notifications.js`
`PUT`	`/api/notifications/read-all`	Mark all read	None	Bearer JWT	`notifications.js`
`GET`	`/api/reports/dashboard-stats`	Aggregated analytics	None	Bearer JWT	`dashboard.js`, `reports.js`
`GET`	`/api/reports/export/cases-csv`	System-wide export	None	Bearer JWT	`reports.js`
`GET`	`/api/users`	List officials	None	Super Admin / SMO	`users.js`
`POST`	`/api/users`	Create official	`{ employeeId, fullName, email, password, role, ... }`	Super Admin	`users.js`
`PUT`	`/api/users/:id/toggle-status`	Toggle account status	None	Super Admin	`users.js`
`GET`	`/api/audit`	Audit ledger	Query params (`action`, `entityType`, `limit`)	Super Admin / Auditor	`auditLogs.js`
`GET`	`/api/config/workflow`	SLA configurations	None	Bearer JWT	`workflowConfig.js`
`PUT`	`/api/config/workflow`	Update SLA rules	`{ slaDays }`	Super Admin	`workflowConfig.js`
`GET`	`/api/health`	Service health status	None	Public	Health monitor
`POST`	`/api/admin/bootstrap-benchmark`	Seed benchmark data	None	Super Admin	Setup console

---

7. Database Architecture & Data Models

Database Provider

- **Engine:** Supabase PostgreSQL
- **Node.js Driver:** pg with the compatibility adapter in backend/lib/pgModel.js
- **Connection Configuration:** Private environment variables in ignored backend/.env; see DEPLOYMENT.md.
- **Offline Behavior:** No in-memory database fallback. Data endpoints return HTTP 503 when PostgreSQL is unavailable.

Document Schemas & Collections

1. `User` (`backend/models/User.js`): Fields: `employeeId`, `fullName`, `email`, `passwordHash`, `role`, `bureauId`, `designation`, `phoneNumber`, `isActive`, `lastLogin`.
2. `Bureau` (`backend/models/Bureau.js`): Fields: `code`, `name`, `description`, `headOfficerName`, `contactEmail`, `jurisdictionScope`, `capacityLimit`, `isActive`.

3. **`Institution`** (**`backend/models/Institution.js`**): Fields: `permanentId`, `name`, `institutionType`, `region`, `state`, `city`, `address`, `pincode`, `affiliatedUniversity`, `approvedIntake`, `contactEmail`, `contactPhone`, `principalName`, `isActive`, `riskScore`.

4. **`Case`** (**`backend/models/Case.js`**): Fields: `caseNumber`, `institutionId`, `permanentId`, `institutionName`, `bureauCode`, `category`, `title`, `description`, `severity`, `priority`, `status`, `assignedOfficerId`, `assignedOfficerName`, `receivedDate`, `slaDeadline`, `isEscalated`, `petitioner`, `timeline`, `remarks`, `documents`, `relatedCaseIds`.

5. **`OverlapReview`** (**`backend/models/OverlapReview.js`**): Fields: `caseAId`, `caseANumber`, `caseATitle`, `caseABureau`, `caseBId`, `caseBNumber`, `caseBTitle`, `caseBBureau`, `institutionId`, `permanentId`, `institutionName`, `confidenceScore`, `layerMatches`, `matchingTokens`, `status`, `reviewedBy`, `reviewerName`, `reviewerRole`, `reviewerNotes`, `reviewedAt`.

6. **`CoordinationRequest`** (**`backend/models/CoordinationRequest.js`**): Fields: `requestingBureauCode`, `targetBureauCode`, `primaryCaseId`, `primaryCaseNumber`, `associatedCaseIds`, `institutionName`, `subject`, `justification`, `status`, `jointActionNotes`, `initiatedById`, `initiatedByName`.

7. **`Notification`** (**`backend/models/Notification.js`**): Fields: `recipientUserId`, `bureauCode`, `title`, `message`, `type`, `caseId`, `linkUrl`, `isRead`.

8. **`AuditLog`** (**`backend/models/AuditLog.js`**): Fields: `userId`, `userName`, `userEmail`, `userRole`, `action`, `entityType`, `entityId`, `details`, `beforeState`, `afterState`, `ipAddress`, `timestamp`.

9. **`SystemConfig`** (**`backend/models/SystemConfig.js`**): Fields: `key`, `value`, `description`.

---

8. Core Regulatory Feature Workflows

1. Automated Overlap Detection Engine (`overlapEngine.js`)

When a case is registered or checked on-demand:

- **Layer 1 (Institution Match):** Filters for existing open cases matching the identical `permanentId` or `institutionId`.
- **Layer 2 (Cross-Bureau Partition):** Restricts candidates to cases lodged under **different** bureaus (`bureauCode !== targetCase.bureauCode`).
- **Layer 3 (Lexical Text Similarity):**
  - Tokenizes title and description into word stems.
  - Removes common English stopwords (`the`, `is`, `complaint`, `college`, etc.).
  - Calculates the Jaccard similarity index:

    `J(A, B) = |A ∩ B| / |A ∪ B|`

  - If `J(A, B) >= 0.25`, an `OverlapReview` docket is generated with confidence score between 50% and 95%.
  - Notifications are dispatched to both involved bureaus.

2. SLA Ageing & Escalation Engine (`escalationEngine.js`)

- Every case is assigned an automatic SLA deadline at creation based on its severity:
  - `CRITICAL`: 7 statutory days
  - `HIGH`: 15 statutory days
  - `MEDIUM`: 30 statutory days
  - `LOW`: 45 statutory days
- Every 30 minutes, `scanAndProcessDeadlines()` executes:
  - Scans all non-disposed cases.

- Flags overdue cases where `currentTime > slaDeadline`.

- Automatically transitions state to `ESCALATED` if overdue criteria are met.

- Dispatches high-priority notifications to bureau supervisors.

3. Institution 360° Consolidated Dossier (`/api/institutions/:id/profile-360`)

Aggregates all multi-bureau history for an institution:

- Computes weighted institutional Risk Score:
  `Risk Score = min(100, (Open Cases 12) + (Critical Cases 25) + (Overdue Cases * 20))`
- Categorizes risk levels:
  - `>= 70`: `CRITICAL / WATCHLIST`
  - `40 - 69`: `ELEVATED INQUIRY`
  - `1 - 39`: `ROUTINE SCRUTINY`
  - `0`: `COMPLIANT / LOW`
- Returns full bureau breakdown, category distribution, and complete chronological case list.

---

9. Error Handling, HTTP 405 Resolution & Security

Root Cause & Permanent Resolution of API Error 405 (Method Not Allowed)

- **The Issue:** When developers launched the frontend using VS Code Live Server (port 5500), `POST /api/auth/login` was sent to `http://127.0.0.1:5500/api/auth/login`. Static file web servers return `HTTP 405 Method Not Allowed` when receiving HTTP `POST` or `PUT` methods.
- **The Permanent Fix:**

  1. `frontend/js/config.js` dynamically detects if the client is running on a port other than 5000 (e.g. 5500 or file preview) and routes all API traffic directly to `http://localhost:5000/api`.

  2. Full CORS preflight handler `app.options('*', cors(corsOptions))` added to Express.

  3. Express `/api/*` fallback safely catches unsupported methods and returns explicit JSON 405 error responses.

  4. Client `ApiClient.request()` checks status 405 and raises informative user guidance instead of crashing.

Security Controls

- **Password Protection:** PBKDF2 / Bcrypt hashing with 10 salt rounds.
- **JWT Protection:** Signed with server-side secret; expired tokens reject with HTTP 401.
- **No Sensitive Leakage:** `passwordHash` stripped on all user queries via `.select('-passwordHash')`.
- **Security Headers:**
  - `X-Content-Type-Options: nosniff`
  - `X-Frame-Options: SAMEORIGIN`
  - `X-XSS-Protection: 1; mode=block`
  - `Referrer-Policy: strict-origin-when-cross-origin`
- **Audit Logging:** Every critical change (logins, case creation, status changes, escalations, coordination requests) is recorded in the `audit_logs` collection.

---

10. Deployment & Environment Configuration

Running Locally

```
# Double-click launcher on Windows:

start.bat



# Or execute via command line:

cd "C:\Users\Yuvraj\OneDrive\Desktop\Case Connect"

npm run seed:demo

npm start
```

Environment Variables (`backend/.env`)

```
PORT=5000

NODE_ENV=development

FRONTEND_URL=*

PGHOST=your-supabase-pooler-host
PGPORT=6543
PGUSER=your-supabase-pooler-user
PGPASSWORD=your-supabase-database-password
PGDATABASE=postgres

JWT_SECRET=generate_a_unique_random_secret_with_at_least_32_bytes

SESSION_EXPIRY=12h
```

---

11. Local Development Accounts

Demo accounts are restricted to local development. Production startup does not create default accounts.

12. Verification & Test Evidence

1. Critical Scenarios Test Suite (`tests/criticalScenarios.test.js`)

- **25 / 25 Critical Scenarios Passed (100% Success Rate)**
- Validates tokenization, Jaccard lexical similarity, state transition machine, SLA overdue detection, statutory ID validation, PostgreSQL model adapters and Supabase schema compatibility.

2. Live Server End-to-End Test Suite (`tests/testLiveServer.js`)

- **Live suite passed (rollback-isolated database run)**
- Validates `/api/health`, static frontend delivery, Super Admin login, invalid credential rejection, registration, duplicate checks, session restoration, private route protection, benchmark data seeding, cross-bureau overlap detection, role-based login checks, and HTTP 405 error formatting.